

Algorithms and Data Structures

13.02.2024

Exercise 1 (1.5 points)

Consider a binary tree whose visits return the following sequences.

```
Pre-order:      A   B   G   I   C   D   E   F   H   L
In-order:       G   I   B   C   A   F   E   H   D   L
Post-order:      I   G   C   B   F   H   E   L   D   A
```

Report the sequence of keys stored on the tree leaves, moving on the tree from left to right. Please report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. The following is an example of the response format: A X C Y etc.

Solution

```
### Tree guess
Node Key Left Right
1      A   2   3
2      B   4   5
3      D   6   7
4      G NULL 8
5      C NULL NULL
6      E   9  10
7      L NULL NULL
8      I NULL NULL
9      F NULL NULL
10     H NULL NULL
```

Response

I C F H L

Exercise 2 (1.5 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=19$. Insertions occur character by character using open addressing with quadratic probing (with $c_1 = 1$ and $c_2 = 1$). Each character is identified by its index in the English alphabet (i.e., $A=1, \dots, Z=26$). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

C U S C U S

Indicate in which elements the last three letters of the sequence are placed, i.e., C, U, and S, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Solution

```
### hash-table
Character keys
Hash table size = 19
Number of keys = 6
Quadratic probing: c1=1.000000, c2=1.000000.
Key: C U S C U S
Key=C k=003 Final=03
Key=U k=021 Final=02
Key=S k=019 Final=00
Key=C k=003 Coll.= 3 Final=05
Key=U k=021 Coll.= 2 Final=04
Key=S k=019 Coll.= 0 Coll.= 2 Final=06
00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18
S - U C U C S - - - - - - - - - - - -
```

Response

5 4 6

Exercise 3 (1.5 points)

Given the following sequence of integers stored in an array, turn it in a heap, assuming to use an array as an underlying data structure. Assume that, in the end, the largest value is stored at the heap's root. Then, execute the first two steps of heapsort on the heap built at the previous step.

10 6 5 7 1 9 3 18 13

Solution

```

### Heap sort
-----
Initial array: 0 1 2 3 4 5 6 7 8
heap-build
-----
heapify      [3<->7]
             10 6 5 18 1 9 3 7 13
heapify      [2<->5]
             10 6 9 18 1 5 3 7 13
heapify      [1<->3] [3<->8]
             10 18 9 13 1 5 3 7 6
heapify      [0<->1] [1<->3]
             18 13 9 10 1 5 3 7 6
heap-sort
-----
heapify      [0<->1] [1<->3] [3<->7]
             13 10 9 7 1 5 3 6 18
heapify      [0<->1] [1<->3]
             10 7 9 6 1 5 3 13 18
heapify      [0<->2] [2<->5]
             9 7 5 6 1 3 10 13 18
heapify      [0<->1] [1<->3]
             7 6 5 3 1 9 10 13 18
heapify      [0<->1] [1<->3]
             6 3 5 1 7 9 10 13 18
heapify      [0<->2]
             5 3 1 6 7 9 10 13 18
heapify      [0<->1]
             3 1 5 6 7 9 10 13 18
heapify      no-swaps
             1 3 5 6 7 9 10 13 18

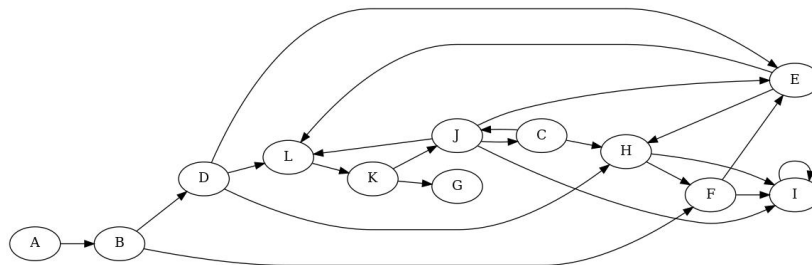
```

Response

10 7 9 6 1 5 3 13 18

Exercise 4 (1.5 points)

Visit the following graph in breadth-first, starting at node A. When necessary, consider nodes and edges in alphabetic order.



Display the minimum distance of all vertices from the starting one. Please indicate the distance of all vertices sorted in alphabetic order (i.e., display the distance for A B C D, etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8, etc.

Solution

```
Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] D [ 1] L [ 2] C [ 3] A [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J [10] K [11] B
Adjacency Matrix:
0 1 0 0 1 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1 0 1 0
0 0 0 0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 1 0 0 0
0 0 0 0 1 0 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 0 1 0 0
0 0 0 0 0 0 0 0 1 0 0
0 1 1 0 1 0 0 0 1 0 0
0 0 0 0 0 0 1 0 0 1 0
1 0 0 0 0 1 0 0 0 0 0
```

```

### Graph visit
BFS:
Node=[ 0] D Discovery_time= 2 Predecessor=[11]
Node=[ 1] L Discovery_time= 3 Predecessor=[ 0]
Node=[ 2] C Discovery_time= 6 Predecessor=[ 9]
Node=[ 3] A Discovery_time= 0 Predecessor=[-1]
Node=[ 4] E Discovery_time= 3 Predecessor=[ 0]
Node=[ 5] F Discovery_time= 2 Predecessor=[11]
Node=[ 6] G Discovery_time= 5 Predecessor=[10]
Node=[ 7] H Discovery_time= 3 Predecessor=[ 0]
Node=[ 8] I Discovery_time= 3 Predecessor=[ 5]
Node=[ 9] J Discovery_time= 5 Predecessor=[10]
Node=[10] K Discovery_time= 4 Predecessor=[ 1]
Node=[11] B Discovery_time= 1 Predecessor=[ 3]
DFS:
List of edges:
[ 3] A -> [11] B : (T) Tree
[11] B -> [ 0] D : (T) Tree
[ 0] D -> [ 1] L : (T) Tree
[ 1] L -> [10] K : (T) Tree
[10] K -> [ 6] G : (T) Tree
[10] K -> [ 9] J : (T) Tree
[ 9] J -> [ 1] L : (B) Back
[ 9] J -> [ 2] C : (T) Tree
[ 2] C -> [ 7] H : (T) Tree
[ 7] H -> [ 5] F : (T) Tree
[ 5] F -> [ 4] E : (T) Tree
[ 4] E -> [ 1] L : (B) Back
[ 4] E -> [ 7] H : (B) Back
[ 5] F -> [ 8] I : (T) Tree
[ 8] I -> [ 8] I : (B) Back
[ 7] H -> [ 8] I : (F) Forward
[ 2] C -> [ 9] J : (B) Back
[ 9] J -> [ 4] E : (F) Forward
[ 9] J -> [ 8] I : (F) Forward
[ 0] D -> [ 4] E : (F) Forward
[ 0] D -> [ 7] H : (F) Forward
[11] B -> [ 5] F : (F) Forward
List of vertices:
[ 0] D: dist_time= 3 endp_time=22 pred=[11] B
[ 1] L: dist_time= 4 endp_time=21 pred=[ 0] D
[ 2] C: dist_time= 9 endp_time=18 pred=[ 9] J
[ 3] A: dist_time= 1 endp_time=24 pred=[-1] -
[ 4] E: dist_time=12 endp_time=13 pred=[ 5] F
[ 5] F: dist_time=11 endp_time=16 pred=[ 7] H
[ 6] G: dist_time= 6 endp_time= 7 pred=[10] K
[ 7] H: dist_time=10 endp_time=17 pred=[ 2] C
[ 8] I: dist_time=14 endp_time=15 pred=[ 5] F
[ 9] J: dist_time= 8 endp_time=19 pred=[10] K
[10] K: dist_time= 5 endp_time=20 pred=[ 1] L
[11] B: dist_time= 2 endp_time=23 pred=[ 3] A
Reachable nodes from vertex [ 3] A:
[ 0] D - [ 1] L - [ 2] C - [ 3] A - [ 4] E - [ 5] F - [ 6] G - [ 7] H - [ 8] I - [ 9] J - [10] K - [11] B -
UN-reachable nodes from vertex [ 3] A:
none

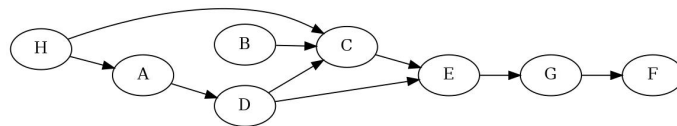
```

Response

0 1 6 2 3 2 5 3 3 5 4 3

Exercise 5 (1.5 points)

Given the following unweighted DAG, find the topological order of its vertices. Start the DFS visit from vertex A. If necessary, consider nodes in alphabetical order.



Report the direct topological order of the vertices. Report vertices on the same line separated by a single space. No other symbols must be included in the response. The following is an example of the response format: F A C H B etc.

Solution

Directed matrix ... stated as such.
 UN-weighted matrix ...stated as such.
 Vertex list: [0] A [1] B [2] C [3] D [4] E [5] F [6] G [7] H
 Adjacency Matrix:
 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 0
 0 0 0 0 1 0 0 0
 0 0 1 0 1 0 0 0
 0 0 0 0 0 0 1 0

```

0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 0
1 0 1 0 0 0 0 0
### Graph DAG
DFS (discovery and end-processing times):
[ 0] A: dist_time= 1 endp_time=12
[ 1] B: dist_time=13 endp_time=14
[ 2] C: dist_time= 3 endp_time=10
[ 3] D: dist_time= 2 endp_time=11
[ 4] E: dist_time= 4 endp_time= 9
[ 5] F: dist_time= 6 endp_time= 7
[ 6] G: dist_time= 5 endp_time= 8
[ 7] H: dist_time=15 endp_time=16
Topological sort (inverse):
[ 5] [ 6] [ 4] [ 2] [ 3] [ 0] [ 1] [ 7]
  F   G   E   C   D   A   B   H
Topological sort (direct):
[ 7] [ 1] [ 0] [ 3] [ 2] [ 4] [ 6] [ 5]
  H   B   A   D   C   E   G   F
Shortest Paths:
  H   B   A   D   C   E   G   F
infy infy 0  1  2  2  3  4
Longest Paths:
  H   B   A   D   C   E   G   F
infy infy 0  1  2  3  4  5

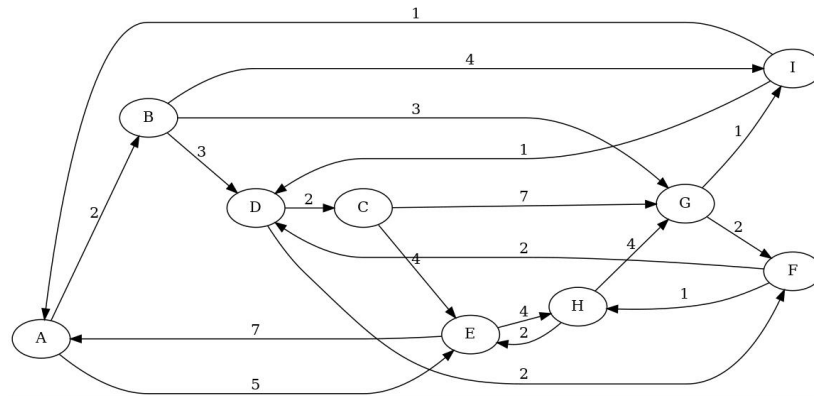
```

Response

H B A D C E G F

Exercise 6 (1.5 points)

Given the following directed and weighted graph, apply Dijkstra's algorithm to find all shortest paths connecting node A with all the other nodes. When necessary, consider nodes and edges in alphabetical order.



Report the shortest paths to all vertices. Please indicate the shortest path to all vertices sorted in alphabetic order (i.e., display the shortest path for A B C D, etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8, etc.

Solution

```

Directed matrix ... stated as such.
Weighted matrix ... stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I
Adjacency Matrix:
0 2 0 0 5 0 0 0 0
0 0 0 3 0 0 3 0 4
0 0 0 0 4 0 7 0 0
0 0 2 0 0 2 0 0 0
7 0 0 0 0 0 0 4 0
0 0 0 2 0 0 0 1 0
0 0 0 0 0 2 0 0 1
0 0 0 0 2 0 4 0 0
1 0 0 1 0 0 0 0 0
### Graph SSSP (Dijkstra and Bellman-Ford)
Dijkstra
Shortest paths from vertex [ 0] A
Node [ 0] A: 0 (pred=-1)
Node [ 1] B: 2 (pred= 0)
Node [ 3] D: 5 (pred= 1)
Node [ 4] E: 5 (pred= 0)
Node [ 6] G: 5 (pred= 1)
Node [ 8] I: 6 (pred= 1)

```

```

Node [ 2] C:  7 (pred= 3)
Node [ 5] F:  7 (pred= 3)
Node [ 7] H:  8 (pred= 5)
Bellman Ford
Iteration 0: A:0 B:2 C:7 D:5 E:5 F:7 G:5 H:8 I:6
Iteration 1: A:0 B:2 C:7 D:5 E:5 F:7 G:5 H:8 I:6
Shortest paths from vertex [ 0] A
Node [ 0] A:  0 (pred=-1)
Node [ 1] B:  2 (pred= 0)
Node [ 2] C:  7 (pred= 3)
Node [ 3] D:  5 (pred= 1)
Node [ 4] E:  5 (pred= 0)
Node [ 5] F:  7 (pred= 3)
Node [ 6] G:  5 (pred= 1)
Node [ 7] H:  8 (pred= 5)
Node [ 8] I:  6 (pred= 1)

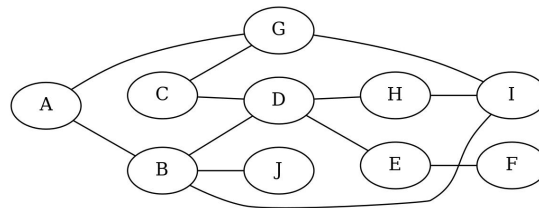
```

Response

0 2 7 5 5 7 5 8 6

Exercise 7 (4.0 points)

Given an undirected graph G , define what bridges and articulation points are. Define (as formally as possible but in plain English) the algorithms usually adopted to find them. Indicate at least one bridge and one of an articulation point in the following graph.



Solution

Please see the teacher's overhead for a complete analysis of the topic.

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

int main(void) {
    char *s1="acdeghjln", *s2="abclmz", *s3;
    char *t1, *t2, *t3;
    s3 = malloc ((strlen(s1)+strlen(s2)+1) * sizeof(char));
    t1 = s1; t2 = s2; t3 = s3;
    while (*t1!='\0' && *t2!='\0') {
        if (*t1 < *t2) {
            *t3 = *t1; t3++; t1++;
        } else {
            *t3 = *t2; t3++; t2++;
        }
    }
    while (*t1 != '\0') {
        *t3 = *t1; t3++; t1++;
    }
    while (*t2 != '\0') {
        *t3 = *t2; t3++; t2++;
    }
    *t3 = '\0';
    printf ("%s", s3);
    return 0;
}

```

Response

aabccdeghjllmnz

Exercise 9 (2.0 points)

The following functions illustrate a possible implementation of quicksort.

```

void quick_sort(int *A, int N) {

```

```

int l, r;
l = 0;
r = N-1;
quick_sort_r (A, l, r);
}

void quick_sort_r (int *A, int l, int r){
    int c;
    if (r <= l)
        return;
    c = partition (A, l, r);
    quick_sort_r (A, l, c);    // LINE 1
    quick_sort_r (A, c, r);    // LINE 2
    return;
}

int partition (int *A, int l, int r ){
    int i, j, pivot;
    i = l-1;
    j = r;
    pivot = A[r];
    while (i<j) {
        while (A[++i]<pivot && i<r);    // LINE 3
        while (j>l && A[--j]>=pivot);    // LINE 4
        if (i < j)
            swap(A, i, j);
    }
    swap (A, x, y);    // LINE 5
    return i;
}

void swap (int *v, int n1, int n2) {
    int temp;
    temp=v[n1];v[n1]=v[n2];v[n2]=temp;
    return;
}

```

Indicate which one of the following statements is correct. Note that incorrect answers imply a penalty in the final score.

1. In the worst case, the time complexity of the algorithm is quadratic in the number of elements of the array.
2. In the worst case, the time complexity of the algorithm is logarithmic in the number of elements of the array.
3. Line 1 and Line 2 must consider the element in position c. In other words, LINE 1 and 2 are correct.
4. In Line 3 the condition $i < r$ is required.
5. In Line 4 the condition $j > l$ is required.
6. In Line 5, `swap (A, x, y)` must be `swap (A, i, r)`.
7. In Line 5, `swap (A, x, y)` must be `swap (A, j, r)`.

Response

1. TRUE.
2. FALSE.
3. FALSE.
4. FALSE.
5. TRUE.
6. TRUE.
7. FALSE.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

void f1 (char *);
void f2 (char *);
int main (void){
    char *s = "ThisIsAString12345";
    f1(s);
    return 0;
}

void f1 (char *s) {
    if (*s=='\0')
        return;
}

```

```

    if (*s>='a' && *s<='z')
        printf ("%c", *s);
    f2 (s+1);
    return;
}

void f2 (char *s) {
    if (*s=='\0')
        return;
    f1 (s+1);
    if (*s>='0' && *s<='9')
        printf ("%c", *s);
    return;
}

```

Response

itig531

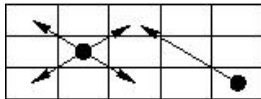
Exercise 11 (3.0 points)

A function receives as parameters two matrices, `matI` and `matO`. The first is the input and the second the output matrix. Both have sizes equal to `R` rows and `C` columns. The function

```
void avg (int matI[R][C], float matO[R][C]);
```

must assign to each element `matO[r][c]` the average of all elements of the primary and secondary diagonals of `matI` centered on the element `matI[r][c]`.

For example, in the following figure



2	3	4	5	0
4	1	7	3	3
2	9	8	1	0

3.67	3.75	2.00	6.00	3.67
5.33	3.40	5.00	3.00	3.00
2.33	6.25	2.80	3.50	2.33

The leftmost matrix represents which elements are considered to compute the average value for the elements in position `[1][1]` and `[2][4]`. The central matrix is a sample of `matI`. The rightmost matrix is the corresponding output matrix `matO`.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define R 3
#define C 5
void avg (int [R][C], float [R][C]);
int main(void) {
    int matI[R][C] = {{2, 3, 4, 5, 0},
                      {4, 1, 7, 3, 3},
                      {2, 9, 8, 1, 0}};

    float matO[R][C];
    avg (matI, matO);
    for (int i=0; i<R; i++) {
        for (int j=0; j<C; j++) {
            fprintf (stdout, "%.2f ", matO[i][j]);
        }
        fprintf (stdout, "\n");
    }
    return 1;
}

void avg (int matI[R][C], float matO[R][C]) {
    int i, j, ii, jj, n;
    float s;
    for (i=0; i<R; i++) {
        for (j=0; j<C; j++) {
            s = n = 0;
            for (ii=i+1, jj=j+1; ii<R && jj<C; ii++, jj++) {
                s = s + matI[ii][jj];
                n++;
            }
            for (ii=i+1, jj=j-1; ii<R && jj>=0; ii++, jj--) {
                s = s + matI[ii][jj];
                n++;
            }
            for (ii=i-1, jj=j-1; ii>=0 && jj>=0; ii--, jj--) {

```

```

        s = s + matI[ii][jj];
        n++;
    }
    for (ii=i-1, jj=j+1; ii>=0 && jj<C; ii--, jj++) {
        s = s + matI[ii][jj];
        n++;
    }
    s = s + matI[i][j];
    n++;
    matO[i][j] = s/n;
}
}
return;
}

```

Exercise 12 (5.0 points)

A string stores a sequence of small letters and decimal digits.

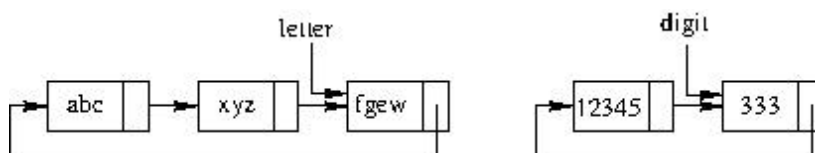
Write the function:

```
void string_to_fifo (char *s, node_t **letter, node_t **digit);
```

which splits the string *s* into subsequences of letters and subsequences of digits, and stores each substring of letters in the FIFO list referenced by the pointer *letter*, and each substring of digits in the FIFO list pointed by *digit*. Both FIFO lists must be circular.

Define the type *node_t* used to implement the nodes of the two lists.

For example, the function must create two lists if the string is *s* = ``abc12345xyz333fgew''. The one referenced by the pointer *letter* storing {abc, xyz, fgew}, and the one referenced by *digit* storing {12345, 333} as represented in the following figure.



Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct node_e {
    char *s;
    struct node_e *next;
} node_t;

void string_to_fifo (char *, node_t **, node_t **);
void enqueue (node_t **, char *);
node_t *new_node ();
// Only for Debug
void traversal (node_t *);

int main (int argc, char **argv) {
    node_t *letter, *digit;

    #if 1
        letter = digit = NULL;
        string_to_fifo ("abc12345xyz333fgew", &letter, &digit);
    #endif
    #if 0
        letter = digit = NULL;
        string_to_fifo (argv[1], &letter, &digit);
    #endif
    // Only for Debug
    fprintf (stdout, "Digit List : ");
    traversal (digit);
    fprintf (stdout, "Letter List: ");
    traversal (letter);
    // Must free letter and digit ... !!!
    return 1;
}

void string_to_fifo (char *s, node_t **letter, node_t **digit) {
    // nld = nothing-digit-letter = 0-1-2
    int i, ndl;

```



```

char *p, *tmp;
tmp = malloc ((strlen(s)+1) * sizeof(char));
if (tmp==NULL) {
    fprintf (stderr, "Allocation error.\n");
    exit (0);
}
ndl = i = 0;
for (p=s; *p!='\0'; p++) {
    if (ndl==0) {
        tmp[i++] = *p;
        if (isdigit(*p))
            ndl = 1;
        else
            ndl = 2;
    } else {
        if (isdigit(*p)) {
            if (ndl==2) {
                tmp[i++] = '\0';
                enqueue (letter, tmp);
                ndl = 1;
                i = 0;
                tmp[i++] = *p;
            } else {
                tmp[i++] = *p;
            }
        } else {
            if (ndl==1) {
                tmp[i++] = '\0';
                enqueue (digit, tmp);
                ndl = 2;
                i = 0;
                tmp[i++] = *p;
            } else {
                tmp[i++] = *p;
            }
        }
    }
}
if (ndl==1) {
    tmp[i++] = '\0';
    enqueue (digit, tmp);
} else {
    tmp[i++] = '\0';
    enqueue (letter, tmp);
}
return;
}

void enqueue (node_t **tail, char *s) {
    node_t *p, *tmp;
    tmp = *tail;
    p = new_node ();
    p->s = strdup (s);
    if (tmp==NULL) {
        tmp = p;
        tmp->next = tmp;
    } else {
        p->next = tmp->next;
        tmp->next = p;
        tmp = p;
    }
    *tail = tmp;
    return;
}

node_t *new_node () {
    node_t *p;
    p = (node_t *) malloc (sizeof (node_t));
    if (p==NULL) {
        fprintf (stderr, "Allocation failed.");
        exit (EXIT_FAILURE);
    }
    return (p);
}

// Only for Debug
void traversal (node_t *tail) {
    node_t *tmp;
    fprintf (stdout, "tail ");
    if (tail == NULL) {
        fprintf (stdout, "-> NULL\n");
    } else {
        tmp = tail->next;
        do {
            fprintf (stdout, "-> %s ", tmp->s);
            tmp = tmp->next;
        } while (tmp != tail->next);
    }
    fprintf (stdout, "\n");
    return;
}

```

Exercise 13 (9.0 points)

Two arrays of float values and length n are given. The first array stores the weights (in kilograms) of a set of n parcels. The second array stores the values (in euros) of the same packages. The parcels have to be uploaded on a truck within its load capacity (we suppose there is no limit in volume). The target is to upload the parcels whose total values (in euros) is maximized.

Write a recursive function

```
void load_truck (float *weight, float *value, int n, float load);
```

receiving as parameters the array `weight`, storing the weights of the packages, the array `value`, storing their cost, `n`, representing the number of packages, `load`, specifying the maximum load capacity of the truck (in kilograms), and it displays the positional number of those parcels that can be loaded to maximize the transported value.

For example, let us suppose that the function is called with the following parameters: `weight = {10.0, 15.0, 25.5, 39.5, 17.0}`, `value = {19.99, 9.50, 15.00, 27.50, 11.40}`, `n = 5`, and `load = 50`. The maximum value that can be transported corresponds to one of the parcels 0 and 3, weight $10.0+39.5=49.5$, and value in euros of $19.99+27.50=47.49$.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

```
#include <stdio.h>
#include <stdlib.h>
// load_truck is directly derived from powerset
void load_truck (float *, float *, int, float);
void powerset_r (float *, float *, int, float, int *, int, int, int, float *, int *, int *);

int main (int argc, char *argv[]) {
    float weight[] = {10.0, 15.0, 25.5, 39.5, 17.0};
    float value[] = {19.99, 9.50, 15.00, 27.50, 11.40};
    int n = 5;
    float load = 50;
    load_truck (weight, value, n, load);
    return EXIT_SUCCESS;
}

// Powerset
void load_truck (float *weight, float *value, int n, float load) {
    int *solution, *best_solution;
    float best_value = 0;
    int best_size = 0;
    int i;
    solution = calloc (n, sizeof (int));
    best_solution = calloc (n, sizeof (int));
    if (solution==NULL || best_solution==NULL) {
        fprintf (stderr, "Allocation error.\n");
        exit (0);
    }
    for (i=n; i>0; i--) {
        powerset_r (weight, value, n, load, solution, i, 0, 0, &best_value, best_solution, &best_size);
    }
    fprintf (stdout, "Solution: ");
    for (i=0; i<best_size; i++) {
        fprintf (stdout, "%d ", best_solution[i]);
    }
    fprintf (stdout, "\n");
    return;
}

void powerset_r (float *weight, float *value, int n, float load, int *solution, int i,
                int pos, int start, float *best_value, int *best_solution, int *best_size) {
    int j;
    float w, v;
    if (pos == i) {
        w = v = 0;
        // DEBUG: printf("{ ");
        for (j=0; j<i; j++) {
            // DEBUG: printf("%d ", solution[j]);
            w = w + weight[solution[j]];
            v = v + value[solution[j]];
        }
        if (w<load && v>*best_value) {
            *best_value = v;
            for (j=0; j<i; j++) {
                best_solution[j] = solution[j];
            }
            *best_size = i;
        }
    }
}
```

```
    }  
    // DEBUG: printf("{} %f %f %f %f\n", w, v, load, *best_value);  
    return;  
}  
for (j=start; j<n; j++) {  
    solution[pos] = j;  
    powerset_r (weight, value, n, load, solution, i, pos+1, j+1, best_value, best_solution, best_size);  
}  
return;  
}
```